AI Case Sorter

Operator documentation

None

Table of contents

1. AI Case Sorter

Desktop app for Windows and Linux that sorts spent brass cartridge casings by **headstamp**. A camera photographs each case, an image classifier predicts the headstamp, and a serial-connected sorting machine drops the case into the right bin.

Two ways to classify:

- **Local model** — a PyTorch ConvNeXt model you trained yourself, downloaded from the community, or imported from a ZIP. Nothing leaves the machine.
- **AI Config** — send the cropped image to an OpenAI-compatible HTTP server.

Community features (model sharing, downloads, the feedback loop) are the only part that needs an account. Everything else runs signed out.

1.1 Start here

The **User Guide** covers every screen, in the order you meet them. It is also the guide the app itself shows on `F1`.

- The window — activity sidebar, status bar, menus
- Panels — serial monitor, classification history, themes
- Sort dashboard — slot cards, templates, running a sort
- Train — capture, label, train
- Models — activate, edit, import, export
- Community — browse and install published models
- Settings — camera, serial, image processing, AI config, theme
- Getting help — support package and updates

1.2 Version dropdown

This site is published per release. The selector in the header switches between them; `latest` always points at the newest stable release, and a release candidate is published under its own version without moving it.

1.3 Download as PDF

This documentation as a single PDF — built per version, so the PDF you download always matches the version you're reading.

1.4 Elsewhere

- Source, issues and releases on GitHub
- Download the latest release
- UI Modernization — the research behind the Qt client
- Contributing and Releasing

2. AI Case Sorter — Qt UI Guide

A guide to the Qt desktop client (launched with `--qt`), written for the person running the machine. It covers every screen and panel you use day to day, in the order you meet them.

2.1 Contents

2.2 How this guide works

This whole guide is one Markdown file, rendered two ways:

- **On GitHub**, browsing straight to `docs/guide/GUIDE.md`.
- **In the app**, in the User Guide panel (`F1`, or Help → User Guide), which opens this file and jumps straight to the section for whatever screen you're on — like Settings → Serial.

Press `F1` again after moving to another screen and the panel follows you. The **< Back** button at the top of the panel returns to the previous section.

2.3 The window

The window is the same everywhere: a column of activity buttons down the left, your working screen in the middle, movable panels around it, a menu bar on top and a status bar underneath.

2.3.1 Activities

The sidebar switches between the things you do, one button each. A thin line splits it in two: the screens that are always live above it, and the pair that follows the active model below it.

Button	What it is
Sort	The Sort dashboard — where sorting happens.
Models	The model library.
Community	The community catalogue. Shown only while signed in.
(separator line)	Below it, the two ways a classifier is taught.
Train	The Train screen — teaching a local model of your own.
AI Config	The AI Config screen — the equivalent step when an HTTP server does the recognising: teaching *it* what to look for.
Settings	Pinned at the bottom, separate from the activities above it: everything you configure once and rarely touch again.

Train and AI Config are always both there, and exactly one of them is in use at a time — which one follows the **active model** (see Models):

- **a model you own** — Train is live, AI Config is dimmed;
- **AI Config mode** ("Use AI Config" on the Models page) — AI Config is live, Train is dimmed;
- **a community model** — neither is live: its publisher trained it, and it, not a server, does the classifying.

Hover either button and the tooltip says which of those you are in. A dimmed button still works — it is the screen behind it that explains the state, and carries a button to the Models page to change it. Never a dead end.

2.3.2 Status bar

Along the bottom, from left to right:

- **Messages** — what the app just did ("Auto-connected to COM3.", "Run stopped."). This is where a refused action explains itself.
- ● **Camera** and ● **Serial** — connection indicators. Green means connected, and the serial one names the port, speed and the firmware version it handshook with.
- **Update to *version*** or **Restart to update** — appears only when there is an app update to fetch or a downloaded one waiting. See updates.
- **Model update: v*N*** — appears when the active community model's publisher has released a newer version. See community model notices.
- **Your name** and **Sign in / Sign out** — the community account. Nothing outside the Community screen needs one.

2.3.3 Menus

- **File → Open data folder** opens the folder holding the database, your models, their training images and the logs. **Quit** closes the app.
- **View** switches each panel on or off, and holds **Re-dock panels**.
- **Help** holds this guide (`F1`), Check for updates..., Export support package..., About and License.

2.4 Panels

Four panels can sit around your working screen. Each one can be moved, tabbed together with another, torn off into its own floating window, or closed:

- **Serial Monitor** — along the bottom, open by default.
- **Classification History** — on the right, closed by default.
- **User Guide** — on the right, closed by default (this guide).
- **Themes** — on the right, closed by default.

Moving a panel: drag it by its *tab* — the small labelled tab at the edge of the panel, not its title. As you drag, blue drop indicators appear showing where it can land: the four edges of the window, or the middle of another panel to tab the two together. Drop it outside the window and it becomes a floating window you can put on a second monitor.

Getting a panel back: View → Re-dock panels returns every open panel to where it started, un-floated. Use it any time a panel ends up somewhere you didn't intend — it always works, which dragging one back does not.

Closing and re-opening: the × on a panel closes it; its entry in the **View** menu switches it back on. Your arrangement is remembered and restored the next time you start the app.

2.4.1 Serial Monitor

Live traffic between the app and the board — every line it sends and every line the board answers, in the order it happened. It is the first place to look when the machine does something unexpected.

- The header shows the connection state, plus **Autoscroll**, **Timestamps** and **Pause**. Pausing holds new lines back and releases them when you un-pause; nothing is lost.
- **Zoom** scales the text from 50% to 200%, for reading the log across the bench.
- **Clear** empties the view. **Save…** writes what you are currently looking at to a text file — filtered lines are not written, so narrow the view first if that's what you want to send.
- **Filter** hides every line that doesn't contain what you type, and the **RX** / **TX** / **Notes** toggles hide a whole direction (received, sent, and the monitor's own commentary). Filtering only hides: clear the box and the traffic is all still there.
- The bottom row sends a command by hand — type it, pick the line ending the firmware expects, press **Send**. `Up` and `Down` walk back through what you have sent before.
- **Baud** changes the port speed and reconnects at it. It is the same setting as Settings → Serial's; change it in either place and the other follows.

2.4.2 Classification History

A grid of tiles, one per sorted case: the cropped image, the headstamp, the confidence, the bin it went to, and a running case number.

Tiles never scroll or move. When the grid is full the newest case overwrites the oldest tile in place, and a coloured border trails the most recent few so you can see where "now" is. That is deliberate — it means you can watch one position and see cases go past, instead of chasing a scrolling list.

Zoom (at the bottom of the panel, 50–200%) sets the tile size. Bigger tiles are easier to read across a bench; smaller tiles mean more of them fit, since the panel holds however many tiles fit its area. Click any tile to open that case's image full size.

2.4.3 Themes panel

The whole list of themes in one place. Click one and the app repaints immediately, so you can try them against the room's lighting without leaving what you were doing. **Edit theme…** opens the theme editor.

This is the same list as Settings → Theme; whichever you use, the other follows.

2.5 Sort dashboard

The Sort activity (the sidebar's top button) is where sorting actually happens. It combines the current case, the slot layout and the run controls on one screen.

On a fresh machine — nothing connected and nothing routed to a slot yet — this screen shows a short panel with buttons straight to Settings → Serial and Settings → Camera instead of an empty grid.

2.5.1 The current case

The left-hand panel shows the **last captured and cropped headstamp** — exactly the image the classifier was given — with what it made of it underneath: the headstamp it matched and how confident it was. A confidence below the confidence floor is coloured as a warning, and that case goes to the Catch-All. Only the current case is shown here; the running record is in the Classification History panel.

Show live camera above the panel adds the live camera feed as a smaller second panel below the crop. It is off by default — the feed is a setup aid, not what an operator watches during a run — and while it is off no frame is fetched or drawn. If the camera isn't running, clicking the feed takes you to Settings → Camera.

2.5.2 Slot cards

Every slot on the machine gets a card, arranged in a grid. Slot 0 is the **Catch-All**, for anything unclassified, below the confidence floor, or not routed to a slot; the rest are your bins. A card shows:

- the slot number (or "Catch-All")
- how many cases have landed there this run
- every headstamp currently routed to it, listed in full — the card grows to fit the list rather than truncating it

Above the grid: **Sorted this run** counts every case this run has sorted, **Reset counts** zeroes the counters (the grid's and the per-bin ones) without touching your assignments, and the template picker names the layout the cards are showing.

2.5.3 Editing an assignment

Click any card except Catch-All to open its assignment editor. Tick a headstamp to route it to that slot; unticking sends it back to the Catch-All. Outside of package mode a headstamp can only be assigned to one slot at a time — ticking it here moves it off whichever slot it was in before, and the row tells you which one that was. A filter box narrows a long headstamp list by name.

2.5.4 Sorting templates

The **Template** picker on the slot grid's header row holds the active **sorting template** — a named snapshot of the whole slot layout, so one model can carry several bin arrangements ("Range brass" vs. "Match prep") and switch between them. **+** creates one (optionally copied from the current layout); ✎ renames or deletes the active one.

You never have to save a template: the active one follows your edits as you make them. Switching templates replaces every slot assignment at once, so the whole picker is blocked while a run is in progress — stop first. Standard and package mode keep separate template lists, because their layouts mean different things.

2.5.5 Run options

⚙ **Run options** on the strip at the foot of the page holds everything that changes how a run behaves:

- **Store images** — keep a copy of each run's captures (none / above the floor / below it / all), under the active model's folder, so AI Config mode stores nothing. Anything but "None" will use real disk space over a long session.
- **Confidence floor** — the percentage a prediction has to reach to be trusted. Below it, the case goes to the Catch-All.
- **Automatically select trays** — when a confident prediction has no slot, route it to the first empty one and keep the assignment.
- **Package mode** and **Batch size** — see package mode.

2.5.6 Start, Stop, and Manual feed

The strip at the foot of the page, with the green **Start** at the far right:

- **Start** begins the continuous sort loop: capture, classify, sort, repeat. It is one button — while the loop runs it reads **Stop**, in red.
- **Stop** ends it. Case counts are kept, not reset, so you can clear a jam and pick back up mid-tray.
- **Manual feed** runs exactly one cycle without starting the loop, useful for testing a single case.

Without a board connected, Start and Manual feed are greyed out and say so. Otherwise starting is refused, with a message explaining why, if the AI Config API key or model name is unset, the active model's checkpoint is missing, PyTorch isn't installed yet for a local model, or a moderator note is waiting to be read.

2.5.7 Package mode

Turning on **Package mode** in Run options switches the grid to batch counting against the one **Batch size** you set, shown as "count / target" on every slot card. A slot that reaches the target stops taking that headstamp; once every slot for a given headstamp is full, the run halts and asks you to empty bins and reset counters. A slot card's ⟲ **Reset** button empties just that bin's counter without stopping the run, so it can keep filling.

2.5.8 Community model notices

When the active model came from the Community and you are signed in, the app checks in with its publisher each time you open the Sort screen. Two things can come back, and neither stops a run in progress:

- **Model update: v*N*** in the status bar — a newer version has been published. The button opens a summary of what you have versus what is available; **Update now** installs it in place, keeping your slot assignments, sorting templates and the name you gave it. Dismissing it is fine — it comes back next time you open Sort.
- **Moderator notes (*N*)** on the run strip — messages from the model's moderators, usually about the feedback images the model is collecting. A note you haven't acknowledged opens by itself and **blocks Start** until you have read it. The same button is the history of every note, including the ones already acknowledged.

If the check can't reach the server, nothing appears and the app behaves exactly as it does offline.

2.6 Train

The Train screen is the loop that builds a training set: feed a case, capture it, label it, save it — and, when you have enough images, train a model from them.

2.6.1 When Train isn't available

Train is always in the sidebar, but it needs a local model of *your own* to work on. When the active model isn't one, the button is dimmed and the page says which of the two cases you are in:

- **AI Config mode** — classification is running over HTTP, so there is no model on this machine to train. Activate or create a local model on the Models page.
- **A community model** — the checkpoint belongs to its publisher and they keep updating it. To build on it, export it from the Models page and import the archive back as your own model.

Either way the page carries a button straight to the model library.

The same holds the other way round for AI Config, which is dimmed whenever a local model is active: clicking it opens its own page, where — in place of the server form — a panel names the model doing the classifying and offers the same jump to the model library. Both buttons stay in the sidebar in every mode — what changes is which one is live.

2.6.2 Capturing images

The left column reads top to bottom in the order you work:

1. The **case image** — the cropped headstamp of whatever was last fed.
2. **Feed**, centred beneath it — drops the next case and captures it. It needs the board connected.
3. **Label** and **Save image** — the label to file this image under, and the button that writes it. The label box remembers every headstamp the model knows, and you can type a new one.
4. The readouts — the predicted label (when a trained checkpoint and PyTorch are both present), how long image processing took, how long the prediction took, and a per-step breakdown. Useful for spotting a setup that has become slow.

PyTorch is *offered* here, never required: capturing and labelling images is exactly the work you do before there is anything to predict with. Declining costs you only the predicted-label convenience, and you won't be asked again this session.

2.6.3 Image counts

The right half lists every headstamp and how many images are on disk for it — your training set, straight from the folder. Drag the divider to give it more width and the list reflows into more columns, which is what makes a model with a hundred-odd headstamps readable.

Clicking a headstamp in this list saves the captured case under that label and immediately feeds the next one — the fastest way to work through a tray of mixed brass.

2.6.4 Training a model

The **Training** strip at the foot of the page turns the images above into a model:

- **Sort while training** routes each case you label into its own bin instead of the catch-all, using the label *you* picked — so a training session also sorts the tray.
- **Training settings…** opens the hyperparameters (epochs, batch size, learning rate, image size, focal loss, SWA and the rest). The defaults are sensible; change them when you know why. The ConvNeXt size is the model's own property — change it in the model editor, not here.
- **Start training** launches the run in a separate process and opens a live console for it. Closing the console mid-run asks first, and confirming cancels the run — the console is the only thing it reports to.

Training needs PyTorch. If it isn't installed, the app offers to install it here.

2.7 AI Config

The alternative to a local model: classification is sent to an OpenAI-compatible HTTP server, and this screen is where that server — and the headstamps it may answer with — is set up. It is Train's mirror in the sidebar: live whenever no local model is active, dimmed when one is.

2.7.1 When AI Config isn't the one classifying

Activate a local model and this screen swaps its form for a panel naming the model that is classifying instead, with a button straight to the Models page. Select **Use AI Config** there to come back — the server settings are still exactly as you left them.

2.7.2 Setting up the server

- **Server** — endpoint URL, API key, model name, the prompt (use `{{headstamps}}` where the list of headstamps should be injected), and the JPEG quality and scale used for the image. Press **Save** to apply — these fields do not save as you type.
- **Headstamps** — the list this mode routes on, and each one's slot. Add, rename, remove, clear, or **Load from server**. These do save immediately, and editing a slot here updates the Sort dashboard's cards straight away.
- **Test shot**, under *Test (capture → crop → classify)* — takes one frame from the camera, runs it through the same pipeline, and shows the label and confidence that came back. No board feed needed; it reads the fields above as they stand, so an API key and model name must be filled in.

2.8 Models

The model library. Everything the app knows how to classify with is a row in this table, plus one synthetic row for AI Config mode.

2.8.1 The model list

The table lists each model's name, whether it is active, its cartridge, type (yours or a community model), the ConvNeXt size it was built as, how many training images it has, whether it has been trained, and when. Click a column heading to sort by it.

The **Active** column marks the model the app currently classifies with: exactly one row reads ● **ACTIVE** in the theme's action colour, and every other row's Active cell is blank. To change it, select a row and press **Activate** at the bottom right.

Above the table: filters by **cartridge** and **type**, a search box, and **New cartridge**, **New model** and **Import…**.

The **"Use AI Config"** row sits at the top of the list, whenever the filters and the search box leave it there. Activating it puts the app in AI Config mode — classification goes to the HTTP server configured on the AI Config screen instead of a local model. It is not a model, so Edit, Delete, Export and the rest stay greyed out while it is selected.

2.8.2 Managing a model

Everything on the bar under the table acts on the **selected** row. **Delete** sits alone on the far left and **Activate** on the far right, so the destructive one and the primary one can never be neighbours:

- **Delete** — delete the model and its folder, after a confirmation. The active model refuses: activate something else first.
- **Edit…** — name, cartridge, model size, primer settings, and the community feedback opt-in where it applies.
- **Images…** — browse the model's training images as thumbnails, and reclassify or delete them. A single image opens full size with prev/next navigation. Models you own only.
- **Headstamps…** — the model's headstamp list: add, rename, delete, set each one's slot, and group headstamps under parent classifications. Renaming a headstamp also renames its training images, so the label and the files stay in step.
- **Evaluate…** — score the model against a folder of labelled images and write an interactive HTML report. Only open reports you generated yourself, from image folders you trust.
- **Export…** — write the model out as a ZIP (checkpoint, headstamps and images), the format the Windows app uses.
- **Activate** — make this the model the app classifies with. Greyed out on the row that is already active.

2.8.3 Importing and exporting

Import… reads a model ZIP back in, after a notice that a model file can execute code — import only from authors you trust. If the archive carries a community ID you already have installed, the app asks whether to **update the installed one in place** — keeping its slot assignments, sorting templates and your name for it — or to **import it as a separate copy**; anything else lands as a new model. A ZIP you import stays *yours*: importing your own model onto a new machine leaves it trainable.

Import and export both run in the background; a model with its images can be large.

2.9 Community

Published models, shared by other users. Signing in is the only thing in the app that needs an account — everything else works signed out.

The table lists each model with its cartridge, version, what the archive includes (model, images, or both), headstamp and image counts, size, publish date, author, and its **State** against your library: Available, Update available, or Installed.

Select a row and the two buttons under the table follow its state. The right-hand one is the primary, and it says what the state makes possible:

Button	Action
Download model	Download and install it, after a notice that a model file can execute code — download only from authors you trust.
Update model	Update your installed copy in place, or take this version as a separate copy; it asks which.
Already installed	Nothing to do — this version is the one you have.
Remove	On the far left, and live only for an installed, current model: delete your local copy. The catalogue entry stays, and the primary goes back to **Download model**.

Select a second model and press Download while one is still running and it queues behind the first, with the status showing "(2 of 3)" as it works through them. Selecting a queued model shows where it sits in the queue, and the one being fetched reads **Downloading…**.

A model you install this way is **managed by its publisher**: it can't be trained here, and updates from them install over it cleanly. The Sort screen tells you when a newer version exists — see community model notices.

Share a model… publishes one of your own models to the community. It appears only for accounts the server grants the contributor role. Sharing does not make the model foreign: your copy stays yours and stays trainable.

2.10 Settings

The Settings activity groups everything you configure once and rarely touch again, behind a section list. Most settings here save as you change them, with no separate Save step. The exception is the Camera page, where the device and resolution are committed by **Apply**.

2.10.1 Camera

Picks which camera the Sort dashboard's preview and the classifier both read from.

- **Detect / refresh** probes attached cameras (opening each one briefly to read its supported resolutions) and fills the **Camera** and **Resolution** dropdowns.
- **Apply** swaps the live camera to the selected device and resolution — the preview beneath updates immediately, so a bad pick is obvious before you leave the page.

Nothing here grabs a camera on its own; both actions are yours. The current device and resolution are shown beneath the controls.

2.10.2 Serial

Connects the app to the sorting machine over the board's UART protocol.

- **Port** — pick a detected serial port, or **Emulated** to run against the built-in board emulator with no hardware attached.
- **Baud** and **probe timeout** — connection parameters; the defaults match the firmware. The baud picker is shared with the Serial Monitor's.
- **Connect / Disconnect** open or close the link; the status bar's serial indicator mirrors the result.
- **Refresh ports** re-scans, for a board plugged in after the app started.
- **Open serial monitor ↗** reveals the Serial Monitor panel.
- **Initialize these settings on startup** pushes the board init settings below automatically on every connect, instead of only when you press "Push to board".

Board init settings covers the machine's tunables — feed and sort speed, homing offsets, motor current, debounce timing and the camera LED level — with **Get config from board / Push to board** to read or write them. The **Sort arm** group jogs the arm to a slot or homes it, for testing wiring before a real run. **Airdrop configuration** holds the three timing values (pre-drop delay, signal duration, post-drop delay), plus the switch that turns it on, for boards fitted with an airdrop mechanism.

2.10.3 Image Processing

Tunes how a captured frame becomes the 480×480 headstamp image the classifier sees. **Capture** takes a frame and shows it beside the processed result, and every detection or primer change re-processes that same frame — so you tune against one case instead of re-feeding for every adjustment.

These settings belong to the active model, not to the app: case diameter and primer size are properties of the cartridge, so switching models brings its own values back. A model that has never been tuned inherits whatever is currently set, so nothing is lost when you activate one for the first time.

- **Configuration** — the circle-detection parameters: accumulator scale, minimum separation between centres, edge strength, detection threshold, and the minimum and maximum case radius in pixels. Start with the two radius values: they bracket the size of the case in the frame.
- **Primer mask** — leave the primer alone, keep only the primer area, or hide it, plus the primer radius. Hiding the primer helps when primer markings confuse the classifier.
- **Camera LED brightness** — the board's ring-light level. This one *is* global: it is a property of the machine, not of the model. The board is updated shortly after you stop moving the slider.

2.10.4 Theme

Picks the colour theme. The same list is in the Themes panel, which is the easier place to try several.

Edit theme… opens the theme editor: it starts from the theme you are currently using and gives you a colour picker per role with a live preview. **Save & apply** writes back to a theme you made (renaming it moves it rather than copying), makes a new one when you started from a built-in, and leaves the editor open so you can keep adjusting — **Close** ends the session; **Create new…** saves under a name you pick, and a built-in is never overwritten either way. A theme can also be exported to a file and imported on another machine.

2.11 Getting help

2.11.1 The guide

`F1`, or **Help → User Guide**, opens this guide in a panel at the section for the screen you are on. It is a normal panel: dock it beside your work while learning something, close it after.

2.11.2 Support package

Help → Export support package… collects your current configuration into a plain-text report — the app version and platform, the active model, run options, training configuration, image processing, serial and camera settings, and the AI Config setup.

Copy to clipboard gives you the text to paste on the community Discord when asking for help. **Save package…** writes a ZIP holding the same report plus a machine-readable `config.json`.

It is safe to share: the API key is reported only as "set" or "not set", nothing is read from the sign-in cache, and file paths are shown relative to the data folder, so your home directory never appears.

2.11.3 Updates

The app checks for a new release shortly after it starts and, if there is one, offers it in the status bar. **Help → Check for updates…** asks immediately.

The dialog shows the release notes and a **Download & install** button. Downloading only stages the update — nothing is replaced until you restart, and the button becomes **Restart now** when it is ready. **Choose a different version…** lists every published release, including older ones, and optionally pre-releases (which the automatic check never offers). You can turn the automatic check off in the same dialog.

3. UI modernization — research & decisions

Working document tracking the investigation into replacing or refreshing the Tkinter UI. Audience: Seth + contributors. Status: **research plus a built, opt-in Qt UI** — the branch carries a full-parity `sorter/qtui/` behind `--qt` (see "Current state" below); whether it *replaces* `sorter/ui/` is still Seth's call, and nothing here retires the Tk UI.

3.1 Problem

The current Tkinter/ttk UI looks dated and has structural ceilings we can't theme our way out of:

- Blurry/limited fractional DPI scaling on Windows.
- Dated widget rendering, especially on Linux.
- Headless testing requires Xvfb (`xvfb-run -a pytest`); without a display the UI test modules skip.
- No animation/layout niceties; a lot of hand-rolled infrastructure (`ScrollableFrame`, `retheme_widgets`, gradient/halftone painting) exists to compensate for the toolkit.

3.2 Requirements

1. Modern, UX-friendly look.
2. Windows + Linux + macOS.
3. **No dependencies outside PyPI** — no system packages, no bundled non-Python runtimes.
4. Headless-testable, ideally without Xvfb.

Port principle: function parity, not UI parity. The end state must cover (more or less) everything the Tk UI does — sorting, models, training, AI config, camera/serial/image-proc setup, community, updates — but the Qt UI is free (and expected) to redesign the UX rather than clone the current screens one-for-one. Concretely: parity is tracked per *capability*, not per tab/dialog; the Tk UI is the behavioral reference (what must be possible), not the layout reference (how it looks). Note the app already deviates from the WinForms original in UI while keeping its behavior — this is the same move again.

3.3 What's at stake (sizing)

`src/sorter/ui/` is ~13,500 lines across 24 modules (largest: `tab_run.py` 1751, `theme.py` 1500, `tab_models.py` 1246, `app.py` 1082), plus ~4,000 lines of UI tests. A toolkit swap is a full rewrite of that layer.

Mitigating factor: the non-UI layers (`hardware/`, `control/`, `data/`, `ml/`, `community/`, `update/`) talk to the UI only through the event bus, so the seam for a swap already exists.

3.4 Options evaluated

3.4.1 1. PySide6 (Qt for Python) — rewrite candidate

- **Pros:** Only option meeting all four requirements. Qt is bundled inside the PyPI wheel (no system Qt) on all three OSes. Modern, DPI-aware widgets; full styling via QSS so the theme system ports. Headless testing is first-class: `QT_QPA_PLATFORM=offscreen` runs real widget tests with no display, and `pytest-qt` is mature. Signals/slots are thread-safe by design and could replace the hand-rolled `EventBus` + 50 ms drain loop. `QImage` renders BGR numpy camera frames directly.
- **Cons:** Full rewrite of `ui/` and its tests. Heavy wheels — measured in the spike: the `PySide6` meta-package is 256 MB download / 648 MB on disk (it drags in QtWebEngine/Qt3D/Charts via `pyside6-addons`, none of it used); `PySide6-Essentials` alone is ~80 MB / ~200 MB and covers everything the app needs. Still small next to torch. LGPL (fine for this project). Prefer PySide6 over PyQt6 (same toolkit, but PyQt6 is GPL/commercial).

3.4.2 2. Stay on Tkinter, modernize the skin — cheap option

- **Pros:** Near-zero migration risk; event bus, threading rules, and all UI tests survive. sv-ttk (Sun Valley / Windows-11-style ttk theme, pure PyPI) or continued investment in our own `theme.py` (already a full theming engine).
- **Cons:** Tkinter's ceiling remains: DPI, rendering, Xvfb-only testing. customtkinter and ttkbootstrap specifically **fight our architecture** — customtkinter replaces ttk widgets with canvas-drawn ones (breaks `retheme_widgets` and the style system), ttkbootstrap wants to own the ttk style engine that `theme.py` owns.

3.4.3 3. NiceGUI / Flet (web-rendered) — rejected

- **Pros:** Easiest modern look; NiceGUI's headless pytest `user` fixture is best-in-class (no browser at all).
- **Cons (deal-breakers):** A native window needs pywebview → system WebKitGTK on Linux, violating requirement 3; otherwise the app lives in a browser tab. Live camera preview becomes MJPEG streaming over localhost. Hardware threads + web event loop is a worse threading story than today. Flet additionally ships a Flutter client binary and has had API churn.

3.4.4 4. Kivy / Dear PyGui — rejected

Kivy: PyPI-pure but mobile-toolkit look on desktop (non-native dialogs, weak menu/DPI story); headless needs GL workarounds. Dear PyGui: GPU required, effectively untestable headless, poor accessibility.

3.4.5 5. wxPython — rejected

No manylinux wheels on PyPI (Linux needs an extra index or a compiler), violating requirement 3. Look is native-2010, not modern.

3.5 Recommendation

PySide6, via an incremental port on a long-lived branch:

1. Spike: shell (main window + status bar) plus one simple tab (Serial or Camera) to prove out theming, threading, and camera-frame rendering.
2. Judge look and per-tab effort from the spike before committing.
3. If accepted, port tabs one at a time; the event bus keeps both UIs drivable during the transition.

Fallback if the rewrite cost is too high now: option 2 (sv-ttk / own theme work) — a visual refresh in days instead of weeks, but the DPI/rendering/Xvfb ceilings remain.

3.6 Spike: PySide6 shell co-existing with the Tk UI

Status: implemented on this branch. Design goals: prove the risky parts (theming, threading, camera rendering, headless tests) **without touching `src/sorter/ui/` at all**, so the branch keeps merging cleanly with upstream `main`'s Tk UI changes and both UIs can be developed in parallel.

How the two UIs co-exist:

- New package `src/sorter/qtui/` beside `ui/` — nothing in `ui/` changes.
- Launch: `python -m sorter --qt` (or `CASESORTER_QT=1`); default launch is the Tk UI, unchanged. The only shared file touched is `__main__.py`, a few-line branch — minimal merge surface against upstream.
- PySide6 is an optional extra (`[qt]`, mirroring how torch is `[ml]`), so end users and CI don't pull the wheels at all — verified: bootstrap's `--no-dev` sync never installs the extra. Dev setup: `uv sync --no-install-project --extra qt` — **add `--extra ml` in the same command if you use local models**: a sync installs exactly the extras named, so syncing `qt` alone removes an installed torch (and a `.python-version` bump recreates `.venv` outright, observed going 3.13→3.14 on Windows). Not data loss — the in-app gate reinstalls torch on the next local-model run — but a ~2 GB redownload waiting to happen.
- Both UIs reuse the non-UI layers unchanged: `EventBus` (Qt drains it with a 50 ms `QTimer` instead of `root.after` — same threading contract), `Camera`, `SerialBroker` / `EmulatorBroker`, `Config` / `SettingsRepo`.
- **One source of truth for colors:** the palettes and the custom-theme registry live in `qtui/palettes.py` — a copy of `ui/theme.py`'s palette half, byte-compared by `test_qt_drift_pins.py` — rendered as QSS by `qtui/theme.py::build_stylesheet`. (It *imported* `sorter.ui.theme` until 2026-08-13; `ui/theme.py` imports tkinter at module level, so a PySide6-only install couldn't launch. Both UIs still read the same `ui.custom_themes` settings row, so a theme built in either shows up in the other.)

Increment 1 scope (its layout is superseded by increment 2 below; everything else carries over): main-window shell (gradient header, theme picker, tabs, status bar), serial/camera status indicators driven by the bus, live camera preview (BGR numpy → `QImage`), serial auto-connect (same port-walking probe as the Tk UI), theme switching with persistence. Run/Serial tabs are placeholders — no tab logic is ported.

Headless testing: `tests/unit/qtui/` runs with `QT_QPA_PLATFORM=offscreen` — real widget construction and event-bus-driven UI updates with no display and no Xvfb. This is the requirement-4 proof.

3.6.1 Spike findings

Implemented 2026-08-12 (~660 lines: `qtui/app.py` 407, `qtui/theme.py` 135, tests 115). All verification green: 17 qtui tests pass **offscreen with `DISPLAY` unset** in 0.24 s (no Xvfb, no pytest-qt); full unit suite 812 passed / 0 failed; ruff and ty clean.

- **Headless testing is better than advertised** — the whole window constructs, the bus drives UI updates, and pixmaps rasterize with no display server. Stronger than the Tk side, whose UI tests need a display.
- **Wheel cost is 3.2× the original estimate** (256 MB / 648 MB on disk via the `PySide6` meta-package — the venv grows ~360 MB → 1.0 GB). Cause: `pyside6-addons` (QtWebEngine, Qt3D, Charts, Multimedia — all unused). **Depend on `PySide6-Essentials` (~80 MB / ~200 MB) for a real port.** End users are unaffected either way (extra never installs for them).
- **The `EventBus` ports unchanged.** Swapping `root.after` for a `QTimer` was the entire threading change; `run_worker`, the serial probe, and the `serial/*` topics were copied verbatim and work.
- **Live theme switching collapses to one call** — `setStyleSheet` on the window re-polishes the whole tree; no `retheme_widgets` equivalent needed. Port convention: express color roles as objectNames (`#action`, `#danger` — the QSS analogue of ttk style names) and keep per-widget stylesheets to a minimum, since those are the only thing a theme switch must re-apply by hand.
- **QSS gotchas found:** `QWidget { background-color }` cascades into child labels (needs `#header QLabel { background: transparent }` under the gradient); `QTabWidget::pane { top: -1px }` closes the seam under the selected tab.
- **Halftone/ink-outline themes have no QSS equivalent.** Comic Book renders flat; the ben-day screen and ink borders would need `QPainter` in a `paintEvent` or a generated tiled pixmap. The flat themes port 1:1.
- **`qtui` still imports tkinter transitively** — the palettes live in `sorter/ui/theme.py`. Harmless during co-existence; the palette-extraction refactor (see below) is the one `ui/` change the design eventually needs. *(Resolved 2026-08-13, without touching `ui/`: `qtui/palettes.py` is a drift-pinned copy instead.)*
- **CI wiring is a decision, not free:** `build.yml` syncs without the extra, so the qtui tests skip there until a job adds `--extra qt`. *(Resolved: the `qtui` job, Linux + Windows, `--extra qt` + `QT_QPA_PLATFORM=offscreen`, deliberately outside the matrix — see CLAUDE.md §8.)*

3.6.2 Second increment: the clean-slate layout (2026-08-12)

The tabbed shell was a straight transcription of the Tk UI; increment 2 replaces it with the shell of the design in "Proposed layout (clean-slate)" below, to judge the *navigation* rather than the widgets. Still a layout spike — real chrome, placeholder content wherever porting tab logic would be needed.

- **Activities sidebar** (fixed 84 px, exclusive `QToolButton`s) driving a `QStackedWidget`: Sort / Train / Models / Community, with Settings pinned at the bottom. Replaces the tab bar — the Tk UI's eight tabs don't fit one row, and half of them are setup, not work.
- **Sort is a dashboard**, not a tab: action row (Start/Stop/Manual feed, disabled — no run controller in the spike) over a `QSplitter` holding the camera preview beside the (unported) slot grid, with a one-line recent- classification strip beneath.
- **Settings is one page with a section list** (Camera, Serial, Image Proc, AI Config, Updates, Theme), which is where the six configuration tabs go. The theme picker moved out of the title bar into Settings → Theme; the header keeps title/subtitle only.
- **Serial monitor is a dock panel** (closable/floatable) instead of the Tk detached `Toplevel` — `View → Serial Monitor` is literally the dock's `toggleViewAction()`. It renders `serial/rx`, `serial/tx` and `serial/note` from the bus into a `QPlainTextEdit` with `setMaximumBlockCount(500)` — the ring buffer the Tk monitor hand-rolls with a deque. (Built on `QDockWidget`; moved to Qt Advanced Docking System later — see the decision log.)
- **Menu bar**: File → Open Data Folder / Quit, View, Help → About. The Tk UI has no menu bar at all; this is where "not a tab, not a button" actions (data folder, updates, about) stop competing for status-bar space.

Verified the same way as increment 1: 28 offscreen tests, full unit suite (823) green, ruff/ty clean.

Gotchas found in increment 2:

- `QAction.menu()` **destroys the menu.** Iterating `menuBar().actions()` and calling `.menu()` on each hands back a *Python-owned* wrapper; when the temporary is collected, shiboken deletes the C++ `QMenu` and the menu bar silently loses that entry (later access raises "Internal C++ object already deleted"). Keep the menus in a dict on the window (`self.menus`) and go through that — in app code and in tests.
- `QWidget`**-selector cascade bites again, harder.** The base `QMainWindow, QWidget` rule paints every descendant, so each new container (`#sidebar`, its `QToolButton`s) needs an explicit background or it fights the surface it sits on. Same class of fix as `#header QLabel`.
- `QDockWidget::title` **is style-able, its buttons are not** (without shipping icons): float/close glyphs come from the platform style, so a dark palette gets platform-colored controls on a themed title bar. Acceptable; icons are a later polish item. *(Resolved by the QtAds move: it ships its own SVG button icons as Qt resources, which theme.py re-declares.)*
- `QSplitter::handle` **needs an explicit** `width`/`height` as well as a background, or the themed handle is invisible.
- Emoji-as-icon in the sidebar works and renders fine offscreen, but real SVG icons (`QIcon`) are the end state — emoji colour is out of the palette's control, which breaks the "every color comes from the theme" rule.

3.6.3 Third increment: the showcase (implemented 2026-08-12)

Makes the Sort dashboard actually sort, so the demo needs no slideware: slot cards with live counts from the real `Config`, the `RunController` wired (same preflights as the Tk Run tab; PyTorch install still deferred to the Tk UI), a live recent-classification feed, and a minimally real Settings→Serial page whose port picker includes **"Emulated"** — the whole demo runs against the `SerialEmulator` with no machine on the bench, over the same code path the real board uses.

Demo script (~2 minutes):

1. `./start.sh` → the familiar Tk UI. Close it. Same command with `--qt` → the new UI. (Point: both ship from one tree; end users see no change.)
2. Sort dashboard: sidebar, live camera preview, slot cards.
3. Settings → Serial → port "Emulated" → Connect → status dot goes green, serial dock streams the handshake.
4. Back to Sort → Manual feed, then Start: cards count up, the recent feed scrolls with confidence coloring, the dock shows each exchange.
5. Switch theme mid-run (Settings → Theme): everything restyles live, including the dock and cards.
6. Close, `git log --oneline`: every increment tested headless (no display server) and green in the same suite as the Tk UI.

Explicitly not in the showcase (parity items for the real port): package mode counters/reset, sorting-template UI, slot-assignment editing (checkbox grid + slot details), auto-select `mode/changed` re-render, wish-list capture, the AI-credentials preflight, cropped-frame preview, Serial-page disconnect/init-settings push, and the PyTorch install dialog (deliberately routed to the Tk UI; `dialog_install_torch`'s `after()`-from-worker pattern must not be copied anyway).

Demo caveat: the emulator removes the *hardware* dependency, not the classifier one. On a fresh DB the app is in AI-config mode and Start will fail at classify time against an unconfigured endpoint — demo with a configured AI endpoint or a local model (checkpoint + torch). Manual feed demos fine with the emulator alone.

Increment-3 findings (54 qtui tests, full suite 849, ruff/ty clean):

- Verified `run/*` payload shapes worth pinning for the port: `run/result` carries `ok` and a `slot` even for a failed cycle — counts must key off `ok`; `run/stopped` fires from the loop's `finally` (also on error and package-halt), so button state must derive from `run/started` / `run/stopped` and never from the click handlers; `run/error` does not imply the run stopped.
- The transparent-children rule is now a rule, not a gotcha: every container with a QSS background (`#header`, `#sidebar`, now `#slotCard`) needs a `... QLabel { background: transparent; }` companion.
- New-widget color roles belong in objectNames (`#slotCount`, `#slotNames`), which the single `setStyleSheet` repaints on theme switch; only colors baked into rich text (the feed's confidence spans, the status dots) need hand re-rendering.
- The emulator path is a genuine end-to-end test: `Emulated` → `EmulatorBroker` → `RunController.cycle_once` → timer-delayed `done` → `run/result` → card count, driven headless in 0.12 s by pumping `bus.drain()` in a bounded poll (`drain_until` in the qtui conftest) — the pattern for every future qtui test involving a worker.
- qtui tests now run against a real `Database` + `Config` on `tmp_path` (shared conftest), not a stub — removed defensive code from the widgets.

3.6.4 Current state (2026-08-14)

Past the spikes: every capability of the Tk UI has a Qt surface, and the shape below is what live-testing rounds with JL and Seth converged on. What changed since increment 3, by surface:

- **Panels, not docks.** Four QtAds panels — Serial Monitor (bottom, open), Classification History, User Guide and Themes (right, closed) — with drop-indicator overlays, tabbing onto an occupied area, `View` toggles, per-tab drag hints, and **View → Re-dock panels** as the escape hatch (Seth floated a panel and couldn't get it back; dragging one home takes dexterity, a menu action doesn't). Layout persists as the manager's own XML in `ui.window_state`.
- **Sort is grounded.** The crop the classifier saw is the primary panel with the live camera an off-by-default toggle beneath it (Seth: the Windows app's layout — the operator watches the *crop* and the call made on it); the current headstamp/confidence sits under it. The grid header row carries "Sorted this run" + Reset + the template picker (they belong with what they count), and one foot strip carries Manual feed / Run options / **Start green at the far right**. Moderator notes and the community model-update prompt ride the same strip and status bar.
- **Train reads as its loop.** One capture column — image → centered Feed → Label + Save → readouts — with the image counts beside it behind a 50/50 splitter (they reflow into columns as it widens, which is what makes a 150-class model readable) and a Training strip at the foot pairing "Sort while training" with Training settings… / Start training. Behind a stack: when the active model isn't trainable here, the page is a guidance panel saying which case it is and jumping to Models.
- **Models and Community share one idiom.** Sortable columns above, one selection-scoped action bar below — destructive far left, the primary far right. Models: Delete … Activate, with the Active column left as a "● ACTIVE" marker, inked in the palette's action colour so activity reads by colour as well as text. Community: Remove plus one state-driven primary (Download / Update / "Already installed"), which carries the whole lifecycle — install → update → remove — and queues a second download behind a running one. Row-embedded controls were tried on both and reverted (see the decision rows).
- **Sidebar.** Sort / Train / Models / Community with Settings pinned below, now inked from hand-authored SVGs by the live palette (the emoji wishlist item, done), plus an **AI Config** activity beside a muted Train in AI Config mode (Seth: "it takes the place of the training screen; it is analogous to training for an LLM") — a page of its own, like Train's, not a Settings section. Neither of the pair is ever hidden — JL didn't discover Train existed — only inked `text_subtle`, with the page behind it saying why.
- **Image processing follows the active model.** The primer-mask and crop (Hough) settings have lived on the model row since the WinForms port; nothing read them. The page now reads/writes the active model and mirrors into `config.image_proc` (the live copy the run reads), with a pristine model row inheriting the global rather than resetting it.
- **Support package** (Help → Export support package…): the redacted configuration as report text to paste on Discord, or a ZIP with a machine-readable `config.json` beside it. API key reported as set/not set, paths relative to the data root, auth cache never read.
- **In-app guide.** `docs/guide/GUIDE.md`, one file rendered by GitHub and by `QTextBrowser` alike, opened at the topic for wherever the user is (F1 / Help → User Guide) in the guide panel. `topic_for` covers every activity and every Settings section; a test pins each answer to a real heading. AI Config is a top-level section of the guide, not a Settings subsection.
- **Conventions settled:** sentence case for controls, Title Case for titles; zoom sliders (50–200%, persisted) on the two log-like panels; the notify/confirm seam on every surface that could open a modal.

3.7 Cost to complete (calibration in progress)

Unit of measure: one *increment* = spec → Opus agent implements → review, verify (pytest/ruff/ty), commit. Measured so far: spike 1 ≈ 8 min agent time / ~30 min wall clock (~660 lines); spike 2 ≈ 9 min / ~30 min (~460 lines); user-found fixes ≈ 5–10 min each (2 so far). The remaining parity work is ~17–25 increments (per-chunk table in the session notes; Sort and Models are the dense ones).

First estimate ranged **15–25 h** on the assumption that dense, behavior-pinned chunks run well above spike velocity. JL disputed that as too high, and spike 3 — the deliberate calibration point, ~3× the size of the earlier increments and wired into the real run path — measured **~10.5 min agent time, ~880 changed lines, 2 implementation passes, ~30 min wall clock**: triple the scope at the *same* wall-clock cost. The assumption was wrong; density is absorbed by the agent, and the wall-clock floor is the spec/review/verify cycle (~20 min) rather than the code volume.

Re-baselined estimate: ~8–12 h of session wall clock — the remaining parity work is roughly 10–14 spike-3-sized increments at ~30–45 min each. Residual risk sits in the deferred-items list above (package mode, template UI, assignment editing, the community/auth surfaces that need the real backend) and in increments that need look-and-feel rounds. The user-paced parts (bench validation against the physical machine) still sit outside these hours and set the calendar time regardless.

3.8 What retiring `ui/` would remove (beyond the directory itself)

Audited 2026-08-12: outside `src/sorter/ui/` and `tests/unit/ui/`, nothing in `src/` imports tkinter or `sorter.ui` except the one launch line in `__main__.py` — the layering is genuinely clean. Dropping `ui/` at the end of a full port also retires:

- **`tests/unit/ui/`** (~4,000 lines) — replaced by offscreen Qt tests.
- **The `--qt` switch itself** — Qt becomes the only path in `__main__.py`.
- **Xvfb everywhere.** `build.yml`'s `xvfb-run -a pytest` and the xvfb apt-installs go; Qt tests run on `QT_QPA_PLATFORM=offscreen` with no display server. The launcher-smoke's `import tkinter` check becomes an `import PySide6` check.
- **The Tcl/Tk constraint on the Python runtime.** uv-provisioned Python was chosen partly because its builds bundle Tcl/Tk (bootstrap.py, CLAUDE.md §2, build.yml comments); that requirement — and re-verifying it on every `.python-version` bump — disappears.
- **Tk-compensation infrastructure** that exists only because Tk lacks the feature: `ScrollableFrame`, `ImagePanel`, canvas gradient/halftone painting, `retheme_widgets` (re-colouring widgets that baked colors in), `markdown_render.py` (252-line hand-rolled markdown→Tk-Text renderer — Qt renders markdown natively), and the `widget.after()` threading gotchas.
- **Pillow from the core dependencies, probably.** Outside `ui/` it's used only by `train_convnext.py` and `eval_report.py`, both of which run under the `[ml]` extra where torchvision already requires Pillow. Qt renders numpy frames directly (`QImage`), so the core-dep slot ships ~10 MB lighter. (Verify the transitive guarantee before actually dropping it.)

One prerequisite before `ui/` can be deleted: the palette data (`THEMES`, `BUILTIN_THEMES`, `normalize_palette`, custom-theme persistence) lives in `sorter/ui/theme.py`, and the Qt spike deliberately imports it from there (single source of truth during co-existence). It must move to a toolkit-neutral module first; only the Tk-rendering half of `theme.py` (ttk styles, fonts, canvas painting) dies with `ui/`.

What stays despite feeling UI-adjacent: `pygrabber` (Windows camera names), opencv, and the `EventBus` (still the UI seam; migrating to signals/slots is a separate, optional step).

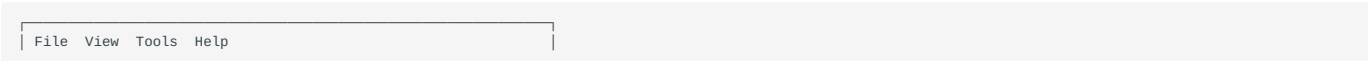
3.8.1 The hardware layer stays toolkit-neutral: pyserial + cv2

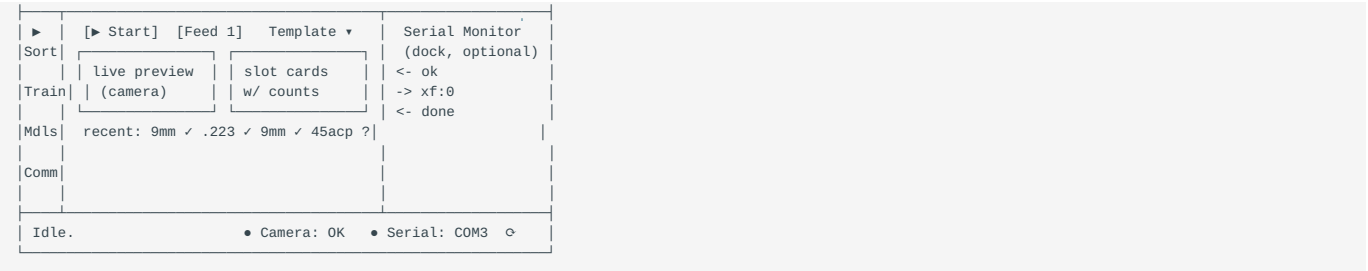
Qt ships its own serial (QtSerialPort) and camera (QtMultimedia) stacks; decided against both:

- **pyserial stays.** The serial consumer isn't the UI — it's the sort loop, a daemon thread making *blocking* calls (`sort_and_move` waits on the board's `done`/`ok` with a timeout). QSerialPort is async, event-loop-driven, and not usable across threads without its own `QThread` +loop, so adopting it means rebuilding the run loop's synchronous waits as state machines — real work, no functional gain. Its enumeration perks (VID/PID, friendly names) pyserial already has. It also lives in `pyside6-addons`, which we deliberately don't install.
- **cv2 stays.** The classifier needs raw numpy BGR frames — exactly what cv2 produces and QtMultimedia doesn't want to hand out.
- Keeping `hardware/` Qt-free is also what lets both UIs share it during co-existence, along with the emulator and the firmware-pinned protocol tests.

3.9 Proposed layout (clean-slate)

Per the port principle above, the Qt UI does not clone the Tk screens. The Tk UI's flat row of eight equal tabs treats daily activities and one-time setup as siblings; actual usage is that **Run is where an operator lives**, Train/Models/Community are occasional, and Camera/Serial/Image Proc/AI Config are setup surfaces visited rarely. The proposed shape follows what mature Qt apps converge on (Qt Creator, OBS, Telegram Desktop, Arduino IDE 2):





1. **Left activity sidebar → `QStackedWidget`**, not a top tab row. Four activities — Sort, Train, Models, Community — plus a Settings entry pinned at the bottom (the Qt Creator mode-selector / Telegram pattern). Mode-driven state (Community shown only when signed in; Train muted rather than hidden when the active model isn't the user's) rides the same `mode/changed` event as today.
2. **Sort is a dashboard, not a form.** Live camera preview *in* the Sort view (operators want to see what the machine sees while it runs), a prominent Start/Stop, the slot-card grid with live counts, and the recent-classification feed integrated — an OBS-style monitoring surface. A `QSplitter` trades preview size against grid size.
3. **All configuration becomes one Settings view**: Camera, Serial, Image Proc, AI Config, Theme as a searchable section list (Qt Creator Options / OBS Settings pattern). Removes four top-level tabs in one move. Updates are *not* settings: they live under **Help → "Check for updates…"** per desktop convention (JL), with the status bar only surfacing a staged update ("Restart to update").
4. **`QDockWidget` for utility panels.** Serial monitor and classification history become dockable/floatable panels — beside Sort on a wide screen, floated to a second monitor, closed when irrelevant. Native Qt strength; the Tk app grew detached toplevels precisely because Tk lacks this.
5. **A real `QMenuBar` + shortcuts** (File/View/Tools/Help): Check for Updates, Open Data Folder, Sign In, About. Free discoverability and accessibility; absent in Tk because Tk menus fight the theming.
6. **Empty states instead of assumptions.** No camera/board/model on first run → guided panels with action buttons where the dashboard will be, not tabs that presume a configured machine.

> Function parity per capability is unaffected — this changes where things live, not what exists. **Spike 2 on this branch implements the shell of this layout** (sidebar, dashboard skeleton, Settings view, serial-monitor dock, menu bar) with placeholders where real tab logic would go.

3.10 Open questions

- Is the pain mostly *visual* (option 2 fixes it) or *structural* — DPI, widget quality, testability (only option 1 fixes those)?
- Keep the `EventBus` as-is under Qt, or migrate to signals/slots? (Bus keeps the non-UI layers untouched; signals are the idiomatic end state.)
- ~~Wheel-size impact on first-run sync~~ — answered: end users never get the extra; devs pay 36 s once. The extra now pins `PySide6-Essentials` (~80 MB; venv 594 MB vs 1.0 GB with the meta-package). Anything from `pyside6-addons` a future feature wants (e.g. QtWebEngine for in-app HTML reports) gets added to the extra explicitly at that point.
- Halftone & ink-outline themes: accept flat rendering under Qt, or invest in a `paintEvent`/tiled-pixmap port? (Spike showed QSS alone can't do it; everything else themes 1:1.)
- ~~CI: add a `--extra qt` job~~ — answered: its own `qtui` job (Linux + Windows, offscreen, no Xvfb), deliberately not a matrix leg, since PySide6-Essentials is one abi3 wheel every leg would re-download.
- **For Seth — Image Proc settings persistence.** The Qt page currently saves each parameter as it changes (no Save button); the Tk/WinForms flow is edit-then-explicit-Save. Options: (a) keep save-on-change — fewer clicks, no "forgot to save" losses, but no way to abandon an experiment; (b) explicit **Save/Revert** pair like the legacy apps — familiar to existing users and gives tuning sessions an undo; (c) save-on-change plus a "Restore defaults / last-saved" escape hatch. Applies to the other settings pages as they land, so worth deciding once.

3.11 Judgment-call register (revisit before the upstream PR)

Every flagged behavior call from the port, in one place (JL asked 2026-08-13). "Kept Tk" = parity preserved; "Changed" = deliberate deviation, revisitable; "Open" = needs a decision.

Call	State	Where
Slot-assign editor moves a headstamp off its old slot in one step (Tk greys it out and needs an untick first)	Changed — same reachable states, one step fewer; "in slot #N" hint shown	#1, `dialog_slot_assign.py`
Settings persistence: Camera/ Image Proc/Serial save-on-change vs Tk's explicit Save (AI Config kept Tk's Save button)	**Open — for Seth** (options a/b/c in "Open questions")	#2/#4 vs #3
Import ZIP offers three-way Update / Copy / Cancel (Tk: update-or-cancel only)	Changed — capability `model_io` always had; adds the "keep both" path	#5, `models_page.py`
`Images…` disabled for foreign/ community models (Tk disables nothing on the Models tab; refusal happened later in Train)	Changed — surfacing the ownership rule earlier	#5, `models_page.py`
Image preview gains prev/next navigation; reclassify/delete keep it open (Tk: one-shot view, closes)	Changed — capability unchanged	#8, `dialog_image_preview.py`
Headstamp rename renames files in `images/` only — not `run_images/` or `feedback_images/`	**Kept Tk — flagged**: feedback queue keeps the old label, which is what gets uploaded; widening is a small follow-up	#17, `dialog_headstamps.py`
`Headstamps…` enabled for every model incl. foreign (renaming a community model's headstamps can break its checkpoint's label mapping)	**Kept Tk — flagged**: alternative is slot-only editing for foreign models	#17
Rename re-syncs the active sorting template (Tk leaves the stale name and drops the slot on next apply)	Changed — fixes a Tk bug	#17
Package-mode slot map (names in a settings key) not rewritten on rename	Kept Tk — same latent issue both UIs	#17
Counters survive Stop/Start; package-halt dialog restored; Clear-all confirm restored	Kept Tk — orchestrator overrides of agent modernizations	#1/#3 overrides
History view: fixed 40-entry cap (Tk: window-size-derived), 3-step recency fade (Tk: 6)	Changed — cosmetic	#13, `history_view.py`
Camera page probes devices on button click, not on page open; device/resolution apply on Apply, not instantly	Changed — no surprise hardware grabs	#2, `settings_camera.py`
Evaluator: Evaluate enabled on checkpoint presence, not ownership (community models evaluable)	Kept Tk	#9
Train Feed button disabled without a board (Tk allows	Changed — Feed follows the connection like Start	#6, `train_page.py`

Call	State	Where
camera-only Feed; the method still works board-less)		
Train counts list: single click saves-and-feeds (same click-intending-selection hazard as Tk's cards)	**Kept Tk — flagged** for a rethink	#6
Training-console close mid-run asks and cancels (Tk closes silently, stranding the subprocess)	Changed — fixes a Tk hazard	#6
Crop + prediction are the Sort page's primary panel; the live camera is an off-by-default toggle (Tk shows the feed)	Changed — Seth, the Windows app's layout; no frame fetched/painted while off	`app.py`
Image-processing settings are per active model, mirrored into the global `config.image_proc` (both UIs previously tuned the global only)	Changed — Seth; a pristine model row inherits the global, so nothing is lost	`settings_imageproc.py`
Community's bar removes the local copy when installed and current (Tk: download only)	Changed — closes the install→update→remove loop. *Was a per-row icon button; the behavior stayed when the trigger moved to the bar*	`community_page.py`
A second download click queues rather than being ignored	Changed — JL; queue position shown as "(2 of N)"	`community_page.py`
AI Config is a sidebar activity that navigates to Settings → AI Config (it has no page of its own)	Changed — Seth; mirrors Train's slot in the other mode	`app.py`
The sidebar is two groups: Sort/ Models/Community, a hairline, then the Train / AI Config pair (Settings still pinned)	Changed — JL; the pair is one choice, and the line says so	`app.py`, `theme.py`
Train and AI Config are both always in the sidebar, muted (not disabled) when not the live mode; the muted one explains itself (Tk hides both)	Changed — JL never discovered Train existed while it was hidden; the mirror is symmetric	`app.py`, `train_page.py`, `settings_ai.py`
~~Models activation is the Active column's radio, not a per-row ✓ button plus "● ACTIVE" text~~	**Reverted 2026-08-14** — JL lived with the row controls and chose the bar; activation is Activate on the selection-scoped bar again (Tk's own idiom), with "● ACTIVE" back as the column's marker	`models_page.py`
~~Both tables carry row-scoped controls (Models ✎/×, Community ↓/↻/×)~~	**Reverted 2026-08-14** — JL, after living with them: one bar per table, scoped to the selection. Everything the experiment brought that isn't the trigger surface stayed (download queue, `installed_state` sync on `models/changed`, the Includes column, full sorting)	`models_page.py`, `community_page.py`
Themes are also a panel, applying on click, alongside Settings → Theme	Changed — trying a theme and configuring one are different activities	`app.py`
		`app.py`

Call	State	Where
Panels are QtAds, with "Re-dock panels" as a guaranteed escape hatch	Changed — stock `QDockWidget` drag-docking was unusable on JL's box	

3.12 Free-hands wishlist

Things the orchestrator would change given a free hand — parked here (JL 2026-08-12) so the port stays conservative. None are commitments; each would be its own proposal:

- **Signals/slots over the polled EventBus.** The 50 ms drain timer is a Tk inheritance; Qt's queued signal connections deliver cross-thread events with no polling, no `max_items` tuning, and type-checked payloads. Big refactor of the seam both UIs share — only sensible after `ui/` retires.
- ~~**Toolkit-neutral palette module.**~~ Done 2026-08-13, but as a drift-pinned *copy* (`qtui/palettes.py`) rather than a shared module, since the real thing means editing `ui/`. Retiring `ui/` collapses the copy back into one module.
- **Typed run events.** `run/*` payloads are ad-hoc dicts ("counts must key off `ok`" is tribal knowledge pinned only by tests); dataclasses would make the contracts self-documenting.
- **A real async classify pipeline.** `RunController` blocks per case (capture → classify → sort serially); overlapping the next capture with the current classify could raise throughput on slow backends. Hardware timing risk — needs Seth.
- **First-run wizard.** A fresh install now lands on a guided empty state (connect a board / connect a camera) instead of an empty grid; a real multi-step wizard through picking a model is still open.
- ~~**SVG icons over emoji glyphs** in the sidebar~~ — done 2026-08-13 (`qtui/icons.py`: one stroke-only SVG per motif, inked from the live palette at render time).
- **Settings search box** (the Qt Creator pattern) once the section count grows past six.
- **Live save-state indicator** on settings pages ("Saved ✓" flash) if the save-on-change model wins with Seth — makes the invisible persistence visible.
- **qtui subpackages, maybe, at the end.** Flat-with-prefixes (`settings_*`, `dialog_*`, `*_page`) matches the Tk `ui/` convention and reads fine at ~27 modules; revisit at increment 16 only if a genuinely cohesive cluster (e.g. community/auth) emerges — group by feature if so, never by widget kind. Not worth the import/blame churn mid-port (JL asked 2026-08-13).
- **Drop the `-100.00%` class of sentinel plumbing**: `api_client` returns `-1` confidence for "server sent none"; an `Optional[float]` would kill a whole family of display guards.

3.13 Windows-app gap analysis (guide v1.1.53) — for the PR

Functionality inventory from the official Application Guide compared against this branch (verified against the Tk source, not assumed). Three buckets:

3.13.1 A. Missing in the whole Python port (Tk AND Qt) — pre-existing gaps

Not qtui regressions: none of these exist in `sorter/ui/` either. Each is a candidate work item; per-item agent tasks in a future increment.

- [ ] **Run modes** (Single/Multi Image Highest Confidence, Highest Average, Popular Highest Average) with **Sample Rotations** count and Serial/Parallel prediction — the multi-rotation ensemble prediction pipeline. The Python port always classifies one image once. (Guide pp. 30–31)
- [ ] **Run speed control** — slider slowing the sort loop.
- [ ] **Model Enhancer** — clone-to-new-model with center images, add primer masks, add rotations, **balance model**, processing-mode conversion, and **binary model creation** (one-vs-rest). (pp. 21–24)
- [ ] **Model Statistics** — confusion matrix on a held-out slice, precision/ recall per headstamp, training time/date/count. The evaluator overlaps but is folder-based; no confusion view exists. (pp. 27–28)
- [ ] **Train screen: Feed Batch** — auto-feed a hopper of same-headstamp brass with a speed slider. (p. 25)
- [ ] **Train screen: UNDO** last added image set. (p. 26)
- [ ] **Train screen: Button Mode automation** chains ([Add]→Feed, [Save]→Add→Feed, ...). (p. 26)
- [ ] **Evaluator: "Improve Training for Anomalies"** (adds rotations for flagged images). (p. 28)
- [ ] **Capture-time rotations** (Use Rotations + rotation count + preview panel) — the port trains ConvNeXt with runtime augmentation instead; functionally overlapping but the guide's explicit-rotations workflow (and Balance-by-rotations) has no equivalent. Decide: adopt or document the difference.
- [ ] **Image-processing detection modes** Manual (click-to-crop X/Y/R) and Hybrid (manual region + auto inside) — the port has Hough only (line-scan ported but hidden). Manual crop is the guide's escape hatch for difficult lighting. (pp. 12–14)
- [ ] **Processing modes** beyond Color (BlackAndWhite, Grayscale, EdgesOnly, Blur, GSBlur) as capture/model settings with their parameter sets. Verify what `model_mode` actually covers in the port; the guide's six modes are richer. (pp. 14–15, 17–18)
- [ ] **Home Feed** button (feed-wheel homing; the port has Home Sorter only).
- [ ] **Advanced Settings free-form key/value grid** for custom firmware parameters (the port has the fixed 14-field list; custom keys can't be sent). (p. 10)
- [ ] **Run screen: Clear Slots** one-click unbind-all (templates partially cover; no explicit clear).
- [ ] **Per-headstamp counters inside each slot** (the port counts per slot total only). (p. 29 #2)
- [ ] **Package-mode alarm parity**: guide beeps 3× and pauses before moving to the next slot; port beeps once. (p. 31)
- [ ] **Config-DB automatic backups** (last 10 retained; corrupted-DB restore prompt). SQLite+WAL is more robust than the JSON store this guarded, but an automatic backup/restore story is still absent. (p. 6)
- [ ] **Serial logging to file** as a launch setting (the Qt monitor's Save… covers manual export; continuous logging doesn't exist).

3.13.2 B. Qt polish gaps (small, from JL's testing + guide details)

- [ ] Image Processing page: label the two previews **Original / Processed** and show **processing time in ms** (guide p. 12; JL).
- [ ] Serial monitor keyboard shortcut (guide: Ctrl+K).
- [ ] Emulation-mode capture parity: guide's emulator serves random sample images so camera-less demo works end to end; port's emulator covers serial only. Consider bundling a handful of sample frames.

3.13.3 C. Equivalent by design (no action; explain in PR)

- Deep-learning toggle (Inception vs ML.Net) ↔ ConvNeXt size choice in training config. Different engines, same knob.
- Sign-in/licensing (guide pp. 3–6) ↔ deliberately absent: OSS build has no license gate; community sign-in is optional.
- Serial monitor, updater, themes, community sharing, sorting templates, wish-list capture: port-side features that exceed the guide.

3.14 Findings for the PR description

- **opencv-python → opencv-python-headless.** The standard wheel bundles its own Qt for `cv2.imshow` (which nothing here calls) and registers its plugin dir on import; those plugins loading against PySide6's Qt libraries caused real rendering corruption (dock float→re-dock artifacts, JL-reproduced). Headless is API-identical, ships no Qt, registers nothing. A launch-site scrub of `QT_QPA_PLATFORM_PLUGIN_PATH` stays as defense for environments with another cv2 on the path.
- **`libxcb-cursor0` becomes a soft Linux dependency.** Qt ≥ 6.5 needs it to load the xcb platform plugin; with the `xcb;wayland` default (floating docks are frozen under native Wayland — upstream Qt), machines without it silently fall back to Wayland and lose dock floating. **Fragmentation policy (JL):** feature-detection with graceful degradation, never distro-detection with hard requirements — the app must launch on any Linux with no new system packages. Plan: (1) runtime check of `QApplication.platformName()`; when Wayland loaded where xcb was preferred, a one-time package-manager-neutral hint (`libxcb-cursor0` on Debian/Ubuntu, `xcb-util-cursor` on Fedora/Arch) pointing at the guide; (2) `bootstrap.py`'s existing probe already handles apt/dnf/pacman with a per-manager package map — adding the cursor lib is one row per manager, ask-first and never blocking; (3) the guide's Linux notes carry the per-family package table (#18 scope). Also verify whether headless opencv still dlopens libGL at all — if not, the probe may shrink instead of grow (#16).

3.15 Decision log

Date	Decision
2026-08-12	Ruled out web-rendered (NiceGUI/Flet), Kivy, Dear PyGui, wxPython against the four requirements. PySide6 identified as the only full-fit; skin-refresh kept as fallback. No go/no-go yet.
2026-08-12	Spike built as a **co-existing** UI: `sorter/qtui/` beside `ui/`, `--qt` / `CASESORTER_QT=1` opt-in, PySide6 as a `[qt]` extra, palettes shared from `sorter.ui.theme`. Lets Qt work proceed in parallel while tracking upstream Tk changes.
2026-08-12	Port principle set: **function parity, not UI parity** — the Qt UI redesigns the UX freely as long as every capability of the Tk UI survives.
2026-08-12	Spike implemented and verified (17 offscreen tests, full suite green, ruff/ty clean). Headless story confirmed; wheel estimate corrected to 256 MB (meta) vs ~80 MB (`PySide6-Essentials` — recommended); halftone/ink themes flagged as the one theming gap.
2026-08-12	`[qt]` extra swapped to `PySide6-Essentials==6.11.1` — pixel-identical (spike uses only QtCore/QtGui/QtWidgets, all in essentials; the meta-package adds only unused addons + stubs). Gotcha for existing dev venvs: the wheels overlap, so uninstalling the meta clobbers `PySide6/__init__.py` and the stubs — fix with `uv sync --extra qt --reinstall-package pyside6-essentials`.
2026-08-12	Hardware layer stays toolkit-neutral (pyserial + cv2); QtSerialPort/QtMultimedia rejected — see "The hardware layer stays toolkit-neutral".
2026-08-12	Clean-slate layout proposed (activity sidebar + Sort dashboard + unified Settings + docks + menu bar — see "Proposed layout"); spike 2 implements its shell.
2026-08-12	Spike 3 (showcase) implemented: Sort dashboard sorts for real — slot cards with live counts, RunController wired, recent feed, Settings→Serial with the Emulated port. 54 headless tests.
2026-08-12	Cost estimate re-baselined from spike 3's measurement: ~8–12 h session time to parity (was 15–25 h; the "dense chunks are slower" assumption measured false).
2026-08-12	**Windows validated**: the showcase build runs on a real Windows machine from a plain `uv sync --extra qt` — sidebar, dashboard and all; only runtime noise is OpenCV's DSHOW "no camera" warning. Requirement 2 now confirmed empirically on Linux + Windows.
2026-08-12	Spike 2 built and verified (28 offscreen tests, full unit suite green, ruff/ty clean). New gotchas: `QAction.menu()` deletes the menu it returns; dock title-bar buttons aren't themable without icons. Sidebar glyphs stay emoji until real `QIcon`s exist.
2026-08-13	**Palettes copied, not imported** (`qtui/palettes.py`, drift-pinned by `test_qt_drift_pins.py`): `ui/theme.py` imports tkinter at module level, so importing it made a PySide6-only install unlaunchable. Custom themes register into the running UI's registry; the shared `ui.custom_themes` row is what still crosses between them.
2026-08-13	**Sort page grounded on the crop, not the feed** (Seth, the Windows app's layout): the cropped headstamp and the call made on it are the primary panel, the live camera an off-by-default toggle (no frame is fetched or painted while off; the grab thread keeps running). History moved wholly into its panel.
2026-08-13	**Vector icons replace the emoji glyphs** — one stroke-only SVG per motif, inked from the live palette at render time (Seth's concept art; Sort/Train carry the machine's own identity). The app/taskbar mark is the one fixed-neutral exception.
2026-08-13	**Model-scoped image processing** (Seth): crop and primer settings follow the active model — the model row has carried them since the WinForms port and nothing read them. Mirrored into `config.image_proc`, which stays the live copy the run reads; a pristine model row inherits the global rather than resetting it. LED brightness stays global (it is a board setting).
2026-08-13	**Support package** (Seth): Help → Export support package… — the configuration as a pasteable report plus a ZIP with a machine-readable `config.json`. Redaction happens at collection time (API key as set/not set, paths relative to the data root, auth cache never read).
2026-08-13	

Date	Decision
	Casing convention: **sentence case for controls, Title Case for window/panel titles** (JL). Zoom sliders (50–200%, persisted per panel) on the classification history and serial monitor.
2026-08-14	**Docks moved from `QDockWidget` to Qt Advanced Docking System** (`pyside6-qtads`, LGPL-2.1+, ~600 KB wheel that pins the same `PySide6-Essentials==6.11.1`). Two rounds of stock-Qt fixes still left drag-docking unusable on JL's Linux box; QtAds brings VS Code-style drop-indicator overlays and lays out in its own splitters. Net *removal*: the whole QMainWindowLayout workaround stack (transition repaint, collapsed-dock floor, 1px resize nudge) is gone — those failure modes don't exist here. API differences that leak: `isClosed()` not `isHidden()`, `setFloating()` takes no argument (re-dock is `addDockWidget`), and `addDockWidget` re-opens a closed panel. Theming is `ads--*` QSS with QtAds's own sheet off (`DisableStylesheet`), which also means re-declaring its button-icon rules. No new system deps (`ldd`: libGL/libxkbcommon/libxcb, all already installed by the qtui CI job).
2026-08-14	**Row actions on the Models table** (JL), matching Community's: ✓/✎/× as fixed-size icon buttons in the row, the AI Config row getting ✓ alone; the bottom bar keeps only the selection-scoped Images…/Headstamps…/Evaluate…/Export…. Community's button carries the full install → update → remove lifecycle, and a second click queues behind the running download. **Superseded the same day — see the revert below.**
2026-08-14	**Foot strips, green primary far right** (JL) on both Sort and Train, with the run counters and the template picker moved onto the slot grid's own header row. Train's capture reads as one column under the image; its counts sit behind a 50/50 splitter and reflow into columns.
2026-08-14	**Themes panel + AI Config activity** (Seth via JL): a panel listing every theme, applying on click and synced with Settings → Theme; an AI Config sidebar entry that takes Train's place in AI Config mode and navigates to its Settings section (it has no page of its own — a second mount would double-parent the widget). **The routing was superseded the same day — see the promotion to a page below.**
2026-08-14	**In-app guide is one file** (`docs/guide/GUIDE.md`), rendered by GitHub and by the guide panel alike, opened at the topic for wherever the user is. `topic_for` covers every activity and Settings section, and a test pins each answer to a real heading so the two can't drift.
2026-08-14	**Sidebar in two groups** (JL): Sort/Models/Community, a palette-driven hairline, then Train and AI Config — both always visible, exactly one live (trainable local model → Train; no active model → AI Config; a community model → neither), tooltips saying which. The muted AI Config now mirrors Train's explainer instead of dumping the user in Settings: it names the active model and jumps to Models. (First as a notice above a greyed form; the page below completes the mirror.)
2026-08-14	**AI Config leaves Settings and becomes an activity page** (JL, live-testing): it was a Settings section that `open_activity` special-cased, so a sidebar click landed the user on *Settings* with the entry unchecked. It is now `qtui/ai_page.py` — a `QStackedWidget` mirroring Train's exactly: the server form when it is the backend, otherwise a full-page explainer naming the model classifying instead plus a jump to Models (the greyed-form-with-a-notice is gone; a form you can't use is noise). `SETTINGS_SECTIONS` no longer lists AI Config, `open_activity` has no special case left, and the guide's AI Config section moved out from under Settings.
2026-08-14	**The Models table's "● ACTIVE" marker is inked in the action colour** (JL, live-testing): activity should read by colour, not only by text. An item foreground brush is baked in and no stylesheet reaches it, so `models_page.apply_palette()` joins the serial log and history cards in `_apply_theme`'s hand re-render list.
2026-08-14	**Row actions reverted; both tables act from a selection-scoped bar** (JL, after living with the experiment above). Models is Delete … Activate with "● ACTIVE" back as the Active column's marker; Community is Remove plus one state-driven primary whose label and role follow the selected row's `installed_state` and the download queue. Only the trigger surface moved: the queue, the `models/changed` state sync, the Includes column and full column sorting all stayed. Net simplification — no item widgets in either table, so nothing has to be rebuilt after a sort or `_pin_ai_row`.